

# AI WEATHER WISE API

## PHASE-WISE PROJECT DOCUMENTATION

**Project Name: AI Weather Wise API**

### Phase 1 – Brainstorming & Ideation Phase

Project Title: AI Weather Wise API

The AI WeatherWise API is a RESTful backend platform designed to provide real-time weather information together with AI-generated summaries and personalized recommendations.

#### Problem Identification

- Users may find it difficult to track multiple favorite weather locations efficiently.
- Raw weather metrics may not provide easy-to-understand guidance.
- Users need personalized recommendations for outdoor activities and clothing.
- A secure system is required to store user preferences and favorite locations.

#### Proposed Solution

The system retrieves weather data, stores user and favorite-location information in MongoDB, and uses Google Gemini AI to generate weather summaries and recommendations.

#### Main Features

- User registration and login with JWT authentication.
- Favorite location management using CRUD operations.
- Real-time weather information retrieval.
- AI-powered weather summaries and recommendations.
- MongoDB database using Mongoose.
- Centralized error handling and request sanitization.
- Fallback mode when external API keys are unavailable.

### Phase 2 – Requirement Analysis Phase

#### Software Requirements

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js v16 or above.
- npm v8 or above.
- Express.js and Mongoose.
- MongoDB.
- Google Gemini AI SDK.
- Postman for API testing.
- Visual Studio Code or similar code editor.

## Hardware Requirements

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum; 16 GB recommended.
- Storage: 1 GB available workspace.

## Functional Requirements

- Register and authenticate users.
- Validate JWT tokens for protected routes.
- Create, read, update and delete favorite locations.
- Fetch current weather metrics.
- Generate AI weather summaries and recommendations.
- Store users and locations in MongoDB.

## Non-Functional Requirements

- Security through password hashing and JWT authentication.
- Maintainability through modular backend architecture.
- Scalability through separated routes, controllers, models and utilities.
- Reliability through error handling and fallback support.

## Phase 3 – Project Design Phase

### System Architecture

The application follows the MVC pattern. The Controller layer coordinates weather retrieval and AI insight generation, while the Model layer manages User and Location entities. The API routing layer acts as the interface in this headless backend.

### Main Components

- API Interface – Postman can send requests to API route paths and receive JSON responses.
- Express Server Gateway – handles HTTP methods, JSON payloads and CORS.
- Authentication Middleware – validates JWT tokens and user authorization.
- Service/Logic Modules – contain application rules and processing logic.
- Database Interface – Mongoose communicates with MongoDB.
- AI Service Integration – Google Gemini generates summaries and personalized recommendations.

### ER Design

User Entity: `_id`, name, email, password, role.

Location Entity: `_id`, user, city, country, createdAt.

Relationship: One User can have many favorite Locations (One-to-Many).

### User Flow

1. User registers or logs in.
2. User saves a favorite city.
3. User requests current weather for the city.
4. User requests AI-generated insights.
5. System validates JWT and returns the requested information.

## Phase 4 – Project Planning Phase

### Development Plan

6. Create the AI Weather Wise project folder.
7. Initialize the Node.js project.
8. Install required dependencies.
9. Configure environment variables.
10. Create the Express server and middleware.
11. Configure MongoDB with Mongoose.
12. Develop authentication and location APIs.
13. Integrate weather and Gemini AI services.
14. Test APIs using Postman or Thunder Client.
15. Document and demonstrate the completed project.

### Project Structure Plan

- index.js – application entry point.
- .env – environment configuration.
- package.json – project dependencies and scripts.
- .gitignore – files excluded from source control.
- Routes, controllers, models, middleware and utility modules – modular backend components.

## Phase 5 – Project Development Phase

### Project Setup

The project is initialized with Node.js. Required packages include Express, Mongoose, bcryptjs, jsonwebtoken, cors, dotenv, multer and the Google Gemini AI package. Nodemon is used for development.

### Environment Configuration

- PORT=5000
- MONGO\_URI=your\_mongodb\_connection\_string
- JWT\_SECRET=your\_jwt\_secret
- GEMINI\_API\_KEY=your\_gemini\_api\_key

### Backend Development

- The index.js file initializes the Express server.
- Environment variables are loaded.
- Middleware is configured.
- MongoDB connection is established.
- API routes are registered.
- Authentication and protected resources are handled through JWT.
- Weather and AI recommendation functionality is exposed through REST APIs.

### Database Development

- User schema stores account details and role.
- Location schema stores favorite city and country information.
- Mongoose manages schemas and MongoDB communication.

## Phase 6 – Project Testing Phase

### API Testing

The document specifies Postman or Thunder Client for testing the REST APIs.

- Register API – tests user registration.
- Login API – tests authentication.
- Weather API – tests current weather retrieval.
- Recommendation API – tests AI-driven recommendation functionality.

### Execution Testing

16. Ensure MongoDB is running.
17. Configure the required environment variables.
18. Run npm install.
19. Start the server using npm start.
20. Verify that the server connects to MongoDB.
21. Send API requests using Postman or Thunder Client.
22. Check returned JSON responses and authentication behavior.

## Phase 7 – Project Documentation Phase

This phase consolidates the project information into a structured document covering the problem, requirements, architecture, database design, development, testing and execution.

### Documentation Contents

- Project description and scenario-based case study.
- Software and hardware requirements.
- System architecture and MVC design.
- ER design and entity descriptions.
- Project setup and environment configuration.
- Backend and database development.
- API execution and testing.
- API testing video and demo references.

### Project Outcome

The project provides a centralized backend service for weather retrieval and AI-powered insights. It supports secure authentication, favorite location management, real-time weather access and context-aware recommendations.

## Phase 8 – Project Demonstration Phase

### Project Execution

23. Open the project in Visual Studio Code.
24. Start MongoDB.
25. Verify the environment configuration.
26. Install dependencies using npm install.
27. Run the backend using npm start.

28. Use Postman or Thunder Client to demonstrate the APIs.
29. Demonstrate Register, Login, Weather and Recommendation APIs.

## Demo References

API Testing Video Link: [API video.mp4](#)

Demo Video Link: [demo video](#)

Code Drive: [Backend-20260925T153057Z-1-001](#)

## Conclusion

AI Weather Wise API is a Node.js and Express.js based RESTful backend that combines MongoDB data management, secure authentication, live weather information and Google Gemini AI insights. The phase-wise documentation presents the project from ideation through demonstration in a structured format.